MUTATION, CLASSES, & ATTRIBUTES Solutions

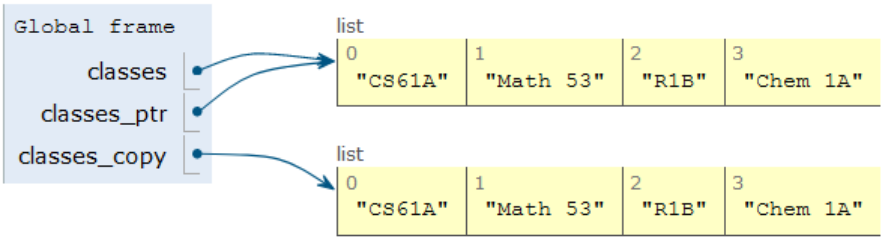
COMPUTER SCIENCE MENTORS 61A

October 12, 2026 – October 16, 2026

1 List Mutation and Environment Diagrams

Let's imagine it's your first year at Cal, and you have signed up for your first classes!

```
>>> classes = ["CS61A", "Math 53", "R1B", "Chem 1A"]
>>> classes_ptr = classes
>>> classes_copy = classes[:]
```

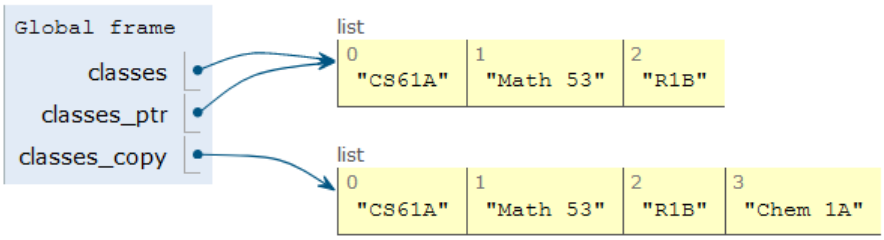


After a few weeks, you realize that you cannot keep up with the workload and you need to drop a class. You've chosen to drop Chem 1A. Based on what we know so far, to change our classes list, we would have to create a new list with all the same elements as the original list except for Chem 1A. But that is silly, since all we really need to do is remove the Chem 1A element from our list.

We can fix this issue with list mutation. In Python, some objects, such as lists and dictionaries, are mutable, meaning that their contents or state can be changed over the course of program execution. Other objects, such as numeric types, tuples, and strings are immutable, meaning they cannot be changed once they are created. Therefore, instead of creating a new list, we can just call `classes.pop()`, which removes the last element from the list.

Created by Cadence Loh, Ana Abell, Tien-Yu Hsu, and Rajoshi Basu. feedback form!
Provided by CSM 61A contributors under CC BY 4.0/3-clause BSD.

```
>>> classes.pop() # pop returns whatever item it removed
"Chem 1A"
```



Here are more list methods that mutate:

Mutability in Lists

Function	Create/Mutate	Action	Return Value
lst.**append**(element)	mutate	attaches element to end of the list	None
lst.**extend**(iterable)	mutate	attaches each element in iterable to end of the list	None
lst **+=** lst2	mutate	attaches lst2 to the end of lst; same as lst.extend(lst2)	None
lst.**pop**()	mutate	removes last element from the list	Removed element
lst.**pop**(index)	mutate	removes element at index	Removed element
lst.**remove**(element)	mutate	removes element from the list	None
lst.**insert**(index, element)	mutate	inserts element at index and pushes rest of elements down	None
lst ***=** n	mutate	attaches lst to the end of lst (n - 1) times; same as calling lst.extend(lst) (n - 1) times	None
lst[start:end:step size]	create	creates a new list that includes elements from *start* index to *end* index - 1 while incrementing *step size*	new list
lst = lst **+** [1, 23]	create	creates a new list with elements from lst and [1, 23]	new list
lst = lst ***** n	create	creates a new list with elements from lst repeated n times	new list
list(iterable)	create	create new list with elements of iterable	new list

1. What would Python display? If an error occurs, write "Error". If a function is displayed, write "Function". If nothing is returned, write "Nothing". Also, draw box-and-pointer diagram for each list created.

```
>>> a = [1, 2]
>>> a[1]
```

```
2
```

```
>>> a.append([3, 4])
>>> a
```

```
[1, 2, [3, 4]]
```

```
>>> b = a[1:]
>>> a[1] = 5
>>> a[2][0] = 6
>>> b
```

```
[2, [6, 4]]
```

```
>>> a.extend([7])
>>> a += [8]
>>> a
```

```
[1, 5, [6, 4], 7, 8]
```

```
>>> a += 9
```

```
TypeError: 'int' object is not iterable
```

Box-and-pointer diagram: https://goo.gl/YJfNgf

2. Draw the environment diagram that results from running the following code.

```
a = [1, 2, [3]]

def mystery(s, t):
    s.pop(1)
    return t.append(s)

b = a
a += [b[0]]
a = mystery(b, a[1:])
```

https://goo.gl/s2XKiG

3. Write a function `insert_n`, which takes in an ascending list of numbers `lst`, a number `x`, and an integer `n`. If `n` is positive, `insert_n` mutatively inserts `n` copies of `x` into `lst` at the correct position so that `lst` is still in ascending order. If `n` is negative, `insert_n` mutatively removes `|n|` copies of `x` from `lst`. (Assume that there are always enough copies to `x` to be removed.)

```
def insert_n(lst, x, n):
    """
    >>> lst = []
    >>> insert_n(lst, 4, 1)
    >>> insert_n(lst, 1, 3)
    >>> insert_n(lst, 2, 2)
    >>> lst
    [1, 1, 1, 2, 2, 4]
    >>> insert_n(lst, 1, -2)
    >>> lst
    [1, 2, 2, 4]
    """
    if n > 0:
        i = 0
        while _____________________________________________:

            ___________________________________________

        ___________________________________________:

            ___________________________________________
    elif n < 0:

        ___________________________________________

            ___________________________________________
```

```
def insert_n(lst, x, n):
    """
    >>> lst = []
    >>> insert_n(lst, 4, 1)
    >>> insert_n(lst, 1, 3)
    >>> insert_n(lst, 2, 2)
    >>> lst
    [1, 1, 1, 2, 2, 4]
    >>> insert_n(lst, 1, -2)
    >>> lst
    [1, 2, 2, 4]
    """
    if n > 0:
        i = 0
        while i < len(lst) and lst[i] < x:
            i += 1
        for _ in range(n):
            lst.insert(i, x)
    elif n < 0:
        for _ in range(-n):
            lst.remove(x)
```

2 Classes and Instances

Object oriented programming is a programming paradigm that organizes relationships within data into **objects** and **classes**. In object oriented programming, each object is an **instance** of some particular class. For example, we can write a `Car` class that acts as a template for cars in general:

```
class Car:
    wheels = 4
    def __init__(self):
        self.gas = 100

    def drive(self):
        self.gas -= 10
        print("Current gas level:", self.gas)

my_car = Car()
```

To represent an individual car, we can then initialize a new instance of `Car` as `my_car` by "calling" the class. Doing so will automatically construct a new object of type `Car`, pass it into the `__init__` method (also called the **constructor**), and then return it. Often, the `__init__` method will initialize an object's **instance attributes**, variables specific to one object instead of all objects in its class. In this case, the `__init__` method initially sets the `gas` instance attribute of each car to 100. It is important to note, however, that you can also manually set object-specific attributes outside of the `__init__` method through variable declaration and methods.

Classes can also have **class attributes**, which are variables shared by all instances of a class. In the above example, `wheels` is shared by all instances of the `Car` class. In other words, all cars have 4 wheels.

Functions within classes are known as methods. **Instance methods** are special functions that act on the instances of a class. We've already seen the `__init__` method. We can call instance methods by using the dot notation we use for instance attributes:

```
>>> my_car.drive()
Current gas level: 90
```

In instance methods, `self` is the instance from which the method was called. We don't have to explicitly pass in `self` because, when we call an instance method from an instance, the instance is automatically passed into the first parameter of the method by Python. That is, `my_car.drive()` is exactly equivalent to the following:

```
>>> Car.drive(my_car)
Current gas level: 80
```

The Same Data with a Dataclass

We can use the same `Car` example with either an explicit initializer or a dataclass. Here we extend the earlier example so construction accepts an owner and an optional starting gas level. These are alternative definitions; use one at a time.

```
class Car:
    wheels = 4

    def __init__(self, owner, gas=100):
        self.owner = owner
```

```
        self.gas = gas

    def drive(self):
        self.gas -= 10
        print("Current gas level:", self.gas)
```

A dataclass declares its instance attributes as annotated fields and generates an `__init__` method automatically. We can define methods in its class body exactly as before:

```
from dataclasses import dataclass

@dataclass
class Car:
    wheels = 4
    owner: str
    gas: int = 100

    def drive(self):
        self.gas -= 10
        print("Current gas level:", self.gas)
```

For this dataclass, the generated initializer takes `(self, owner, gas=100)` and assigns `self.owner` and `self.gas`, just like the explicit initializer above. The unannotated `wheels = 4` remains a class attribute, not a dataclass field or constructor argument. The default for `gas` lets us omit that argument.

The following works with either definition.

```
>>> my_car = Car("Ada")
>>> other_car = Car("Grace", 50)
>>> my_car.owner, my_car.gas, other_car.gas
('Ada', 100, 50)
>>> my_car.drive()
Current gas level: 90
>>> Car.drive(my_car)
Current gas level: 80
>>> other_car.gas
50
```

In either style, calling the class constructs an instance; Python supplies `self` automatically during initialization. Methods also work the same way: `my_car.drive` is a bound method, and `my_car.drive()` has the same effect as `Car.drive(my_car)`. A dataclass method can accept additional arguments, return a result, or mutate instance attributes. Here, `drive` changes the gas level of just the instance passed as `self`. The class object and each instance are objects with their own attributes. Dataclasses also generate conveniences such as a representation and field-based equality; they are still classes.

4. **Museum tickets.** Predict each result, executing the prompts in order. Draw the instances and the references to them as you trace.

```
from dataclasses import dataclass

@dataclass
class Ticket:
    exhibit: str
    remaining: int = 2

    def admit(self):
        if self.remaining > 0:
            self.remaining -= 1
            return True
        return False

original = Ticket("Space")
alias = original
separate = Ticket("Space", 1)
```

```
>>> original.remaining, separate.remaining

(2, 1)

>>> alias.admit()

True

>>> original.remaining, separate.remaining


(1, 1)

>>> original is alias, original is separate


(True, False)
```

Equal field values do not make two instances the same object.

```
>>> alias = Ticket("Ocean")
>>> alias.remaining = 5
>>> original.exhibit, original.remaining, alias.remaining
```

`('Space', 1, 5)`. Reassigning `alias` makes it refer to a new instance; the later attribute assignment changes that new instance.

```
>>> original.admit(), original.admit(), original.remaining

(True, False, 0)
```

3 Attribute Lookup, Assignment, and Bound Methods

5. Consider only the following class; no inheritance is involved.

```
class Account:
    interest = 2

    def __init__(self, owner, balance=0):
        self.owner = owner
        self.balance = balance

    def deposit(self, amount):
        self.balance += amount
        return self.balance

a = Account("Ada", 10)
```

For each expression, write its value or the kind of object it produces. If an error occurs, write **Error**. Execute the expressions in order.

(a) `Account.interest, a.interest`

`(2, 2)`

(b) `Account.deposit`

A function object stored as a class attribute.

(c) `a.deposit`

A bound method object: `Account.deposit` with `a` already bound to `self`.

(d) `a.deposit(5)`

`15`

(e) `Account.deposit(a, 5)`

`20`

(f)
```
>>> a.interest = 7
>>> Account.interest = 4
>>> a.interest, Account("Bob").interest
```

`(7, 4)`. The first assignment writes an instance attribute on `a`; the second writes an attribute on the class object. The instance attribute shadows the class attribute during lookup on `a`.

(g) Explain how Python evaluates `a.deposit(10)`.

First evaluate `a.deposit`. Attribute lookup checks `a` and then `Account`, where it finds the function `deposit`. Looking up that function through an instance produces a bound method with `a` bound to its first parameter. Python then calls it with 10. The equivalent function call is:

`Account.deposit(a, 10)`

6. **Which counter changes?** Execute the prompts in order. Write each displayed value and draw separate attribute tables for the class and its two instances. If an expression raises an error, write **Error** and continue with the next prompt.

```
class Counter:
    step = 1

    def __init__(self, value):
        self.value = value

    def advance(self):
        self.value += self.step
        return self.value

left = Counter(0)
right = Counter(10)
saved = left.advance
```

(a)
```
>>> Counter.step = 3
>>> left.step = 2
>>> left.step, right.step, Counter.step
```

`(2, 3, 3)`. Only `left` has an instance attribute named `step`; `right` finds the class attribute.

(b)
```
>>> saved()
```

```
2
```

```
>>> Counter.advance(right)
```

```
13
```

The saved bound method supplies the original `left` instance as `self`. The class function requires an explicit instance.

(c)
```
>>> left = right
>>> saved()
```

```
4
```

```
>>> left.value, right.value
```

```
(13, 13)
```

Reassigning `left` does not change the instance held by `saved`. Both names now refer to the second counter, while the bound method still refers to the first.

4 Tree Recursion and Result Lists

For the following question, use this dataclass-based tree. A `Tree[T]` has a `label` of type `T` and a list of `branches`, each of which is also a `Tree[T]`. A tree is never empty.

```
from __future__ import annotations
from dataclasses import dataclass, field

@dataclass
class Tree[T]:
    """A Tree has a label (of type T) and a list of branches,
    which are trees.

    >>> t = Tree(3, [Tree(1, []), Tree(2, [Tree(1, []), Tree(1, [])])])
    >>> t.branches[0]
    Tree(label=1, branches=[])
    >>> len(t.branches)
    2
    >>> t.branches[1].label
    2
    >>> len(t.branches[1].branches)
    2
    >>> expr = Tree(10, [Tree(6, [Tree(2), Tree('*'), Tree(3)]),
    ...                  Tree('+'), Tree(4)])
    >>> print(expr)
    10
      6
        2
        *
        3
      +
      4
    """
    label: T
    branches: list[Tree[T]] = field(default_factory=list)  # branches
       defaults to []

    def __str__(self):
        return format_tree(self)

def format_tree(t: Tree, indent='') -> str:
    "Format a tree with each branch indented below its label."
    assert isinstance(t, Tree), f'{t!r} is not a Tree'
    assert isinstance(t.branches, list), f'branches of {t!r} is not a list'
    string = indent + str(t.label)
    for b in t.branches:
        string += '\n' + format_tree(b, indent + '  ')
    return string

def is_leaf(t: Tree) -> bool:
    "Return whether a Tree t is a leaf with no branches."
```

```
    return not t.branches
```

7. **Labels at a depth.** Implement `labels_at_depth`, which takes a tree `t` with integer labels and a nonnegative integer `k`. Return a new Python list containing the labels exactly `k` edges below the root, in left-to-right branch order. The root is at depth 0. If no nodes exist at that depth, return an empty list.

   Use recursion and `extend` to add each recursive result to the local `result` list. Do not change any tree labels or branches.

```
def labels_at_depth(t: Tree, k: int) -> list[int]:
    """
    >>> t = Tree(1, [Tree(2, [Tree(4), Tree(5)]),
    ...               Tree(3, [Tree(6)])])
    >>> labels_at_depth(t, 0)
    [1]
    >>> labels_at_depth(t, 1)
    [2, 3]
    >>> labels_at_depth(t, 2)
    [4, 5, 6]
    >>> labels_at_depth(t, 3)
    []
    >>> labels_at_depth(Tree(9), 1)
    []
    """
    if ____________________________________:

        return ________________________________

    result = []

    for b in ____________________________________:

        result.extend(____________________)

    return result
```

```
def labels_at_depth(t: Tree, k: int) -> list[int]:
    if k == 0:
        return [t.label]
    result = []
    for b in t.branches:
        result.extend(labels_at_depth(b, k - 1))
    return result
```

At depth 0, return a one-element list containing the root's label. Otherwise, the requested nodes are at depth `k - 1` within each branch. Each call creates its own result list. `extend` mutates that list, but the input tree is only read. At a leaf with positive `k`, the loop has no iterations, so the result is empty.

Explain what would change if `extend` were replaced by `append`. Why would **`return`** `result.extend(...)` be wrong?

`append` adds each recursive result as a single list element, introducing nested lists. For example, at depth 1 the result would be `[[2], [3]]`. `extend` adds the elements individually. It returns `None`, so we must return `result` separately.